SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	G.BHARATHI	**Reg. No.:**	192371079
Date:	22-08-2026	**Duration:**	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

1. **Problem Overview**

- Explain the assigned problem statement and summarize the implemented solution.
- Describe the project objectives and key functionalities.

2. **Repository Organization**
    - Evaluate the repository structure, folder organization, and file naming conventions.
    - Justify how the repository layout supports maintainability and collaboration.
3. **Code Quality Review**
    - Review the source code for readability, modularity, coding standards, and documentation.
    - Identify strengths and areas for improvement.
4. **Version Control Evaluation**
    - Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
    - Explain how version control supports collaborative software development.
5. **Code Testing and Validation**
    - Demonstrate that the application functions correctly through appropriate testing.
    - Present test cases, execution results, and screenshots as evidence.
6. **Repository Documentation**
    - Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.
    - Suggest improvements to enhance usability and maintainability.
7. **Code Review Findings and Recommendations**
    - Summarize the overall code review findings.
    - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link

- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15%)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages,	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.

	and repository management practices.			
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15%)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.
Review Findings, Recommendations & Presentation(10%)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5

Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

COLLEGE ADMINISTRATION AND MANAGEMENT SYSTEM

1. PROBLEM OVERVIEW

1.1 Problem Statement

The College Administration and Management System is developed to simplify and manage the major administrative activities of a college through a centralized software application. The system manages student details, faculty information, departments, courses, admissions, attendance, fees, and reports.

The system reduces manual paperwork, minimizes errors, improves data accuracy, and provides quick access to important college information.

1.2 Implemented Solution

The proposed system provides a web-based platform for managing college administration activities. Administrators can add, update, view, and manage student, faculty, admission, attendance, course, and fee information.

The application uses a structured frontend, backend, and database architecture. Git is used for version control and collaborative development, while Docker is used for containerization and deployment.

1.3 Project Objectives

The main objectives of the project are:

- To provide centralized college administration.
- To manage student and faculty information.
- To manage student admissions and enrollment.
- To maintain department and course details.
- To record and monitor student attendance.
- To manage student fee information.
- To generate administrative reports.
- To reduce manual work and paperwork.
- To improve data accuracy and accessibility.
- To provide a maintainable and scalable application.

1.4 Key Functionalities

The major functionalities of the system include:

1. Student Management
2. Faculty Management
3. Department Management
4. Course Management
5. Admission Management
6. Attendance Management
7. Fee Management
8. Examination Management
9. Notification Management
10. Report Generation

2. REPOSITORY ORGANIZATION

2.1 Repository Structure

The project repository is organized into separate folders based on application functionality.

College-Administration-Management/

```
│
├── frontend/
│   ├── public/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── services/
│   │   └── App.js
│   └── package.json
│
├── backend/
│   ├── controllers/
│   ├── models/
```

```
│   ├── routes/
│   ├── middleware/
│   ├── config/
│   ├── server.js
│   └── package.json
│
├── database/
│   └── schema.sql
│
├── docs/
├── screenshots/
│
├── Dockerfile
├── docker-compose.yml
├── .gitignore
├── .env
└── README.md
```

2.2 Folder Organization

The frontend folder contains the user interface and frontend components.

The backend folder contains server-side logic, APIs, controllers, models, routes, and middleware.

The database folder contains database schemas and database-related files.

The docs folder contains architecture diagrams, workflow diagrams, and project documentation.

The screenshots folder contains screenshots used as evidence for Git, Docker, testing, and application execution.

The Dockerfile contains instructions for creating the application container.

The Docker Compose file defines and manages the application services.

The README.md file contains project information, installation instructions, usage instructions, and dependencies.

2.3 Repository Maintainability

The repository structure supports maintainability because each component is organized separately. Developers can easily locate source code, configuration files, database files, and documentation.

The separation of frontend and backend also supports collaboration because different developers can work on different parts of the application independently.

Repository Evaluation

Criteria	Observation
Folder Organization	Well structured and modular
File Naming	Meaningful and descriptive
Component Separation	Frontend and backend are separated
Documentation	README and documentation are included
Maintainability	Easy to modify and extend
Collaboration	Suitable for collaborative development

Screenshot: Paste the screenshot of the GitHub repository structure here.

3. CODE QUALITY REVIEW

3.1 Code Readability

The source code is organized using meaningful file, function, class, and variable names. Proper indentation and consistent formatting make the source code easier to understand and maintain.

Examples of meaningful component names include:

StudentController

FacultyController

AdmissionController

AttendanceController

FeeController

These names clearly describe the purpose of each component.

3.2 Modularity

The application follows a modular structure. Different functionalities are separated into individual components.

For example:

Student Management

↓

Student Controller

↓

Student Model

↓

Student Database

This approach allows individual modules to be modified without affecting the entire application.

3.3 Coding Standards

The following coding practices are followed:

- Meaningful variable and function names.
- Consistent indentation.
- Modular source code.
- Proper separation of frontend and backend logic.
- Reusable components.
- Appropriate comments for complex operations.
- Proper error handling.
- Organized API routes.

3.4 Documentation

The README file provides information about the project, installation process, dependencies, execution commands, and project functionality. Comments can also be used in complex sections of the source code to improve understanding.

3.5 Strengths

The major strengths identified during the code review are:

- Clear project organization.
- Modular code structure.
- Meaningful naming conventions.
- Separation of frontend and backend.
- Reusable components.

• Easy-to-understand folder structure.
• Git-based version management.
• Docker-based deployment support.

3.6 Areas for Improvement

The following improvements can be made:

• Increase unit test coverage.
• Improve error-handling mechanisms.
• Add comments for complex functions.
• Implement automated code quality checking.
• Improve security of environment variables.
• Add input validation for forms.
• Remove unused dependencies and code.
• Implement continuous integration.

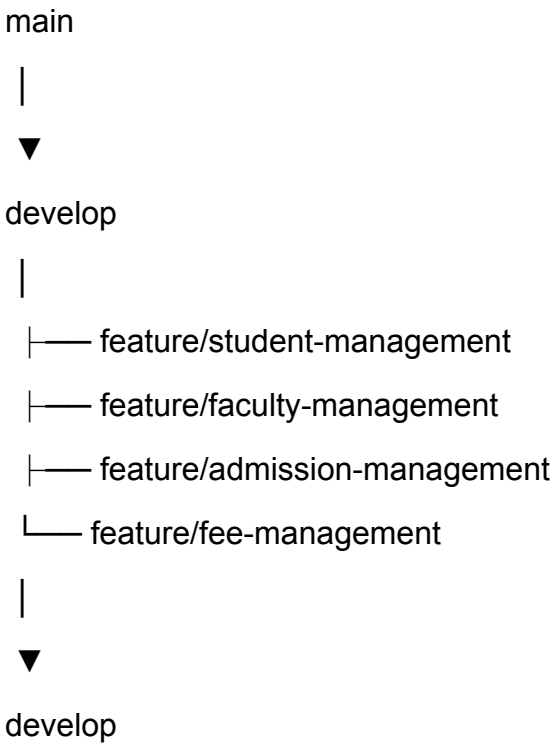
4. VERSION CONTROL EVALUATION

4.1 Git Repository

Git is used as the version control system for the project. It maintains the history of source-code changes and supports collaborative development.

4.2 Branching Strategy

The project follows a structured branching strategy.



release

main

The main branch contains stable code. The development branch contains ongoing development work, while feature branches are used for developing individual functionalities.

4.3 Commit Messages

Meaningful commit messages are used to describe project changes.

Examples:

Initial project setup

Add student management module

Add faculty management module

Implement admission management

Add attendance management

Implement fee management

Add database configuration

Create Dockerfile

Add Docker Compose configuration

Update README documentation

Fix authentication issue

Meaningful commit messages make the repository history easier to understand and maintain.

4.4 Version Control Practices

The following Git practices are followed:

- Regular commits.
- Meaningful commit messages.
- Feature-based branches.
- Pull and push operations.

• Branch merging.
• Conflict resolution.
• Remote repository synchronization.

4.5 Collaboration

Git supports collaborative software development by allowing multiple developers to work on different branches. Developers can create feature branches, implement changes, commit them, and merge them into the development branch after testing.

Screenshot: Paste the screenshot of the Git commit history here.

Command used:

git log --oneline --graph --all

5. CODE TESTING AND VALIDATION

Testing is performed to verify that the application functions correctly and satisfies the specified requirements.

5.1 Test Cases

Test Case	Test Description	Expected Result	Status
TC01	Admin login with valid credentials	Admin dashboard opens	Pass
TC02	Add a new student	Student is successfully added	Pass
TC03	Update student details	Student information is updated	Pass
TC04	Add faculty details	Faculty record is created	Pass
TC05	Add course details	Course is successfully added	Pass
TC06	Register new admission	Admission record is created	Pass
TC07	Record attendance	Attendance is stored successfully	Pass
TC08	Add fee payment	Payment information is recorded	Pass
TC09	Generate report	Required report is displayed	Pass
TC10	Invalid login	Error message is displayed	Pass

5.2 Testing Procedure

The following testing procedure is followed:

1. Start the application.
2. Open the login page.
3. Enter valid administrator credentials.
4. Verify the administrator dashboard.
5. Add a student record.
6. Update student information.
7. Add faculty and course details.
8. Test admission management.
9. Record attendance.
10. Test fee management.
11. Generate reports.
12. Verify that the information is correctly stored and displayed.

5.3 Validation

The application is validated by checking:

• Functional correctness.
• User input validation.
• Database connectivity.
• API responses.
• Data storage and retrieval.
• Error handling.
• User interface functionality.

Screenshots to add:

• Login page
• Admin dashboard
• Student management page
• Faculty management page
• Admission page
• Attendance page
• Fee management page
• Report page
• Successful application execution

6. REPOSITORY DOCUMENTATION

6.1 README Evaluation

The README file provides the basic information required to understand and execute the project.

It contains:

• Project title.
• Project description.
• Project objectives.
• Major features.
• Technologies used.
• Project structure.
• Installation instructions.
• Execution commands.
• Docker instructions.
• Database configuration.
• Usage instructions.

6.2 Installation Instructions

The repository can be cloned using:

git clone <repository-url>

cd College-Administration-Management

Dependencies can be installed using:

npm install

The application can be started using:

npm start

For Docker deployment:

docker compose build

docker compose up -d

6.3 Dependencies

The project uses the following technologies:

• React.js – Frontend development.
• Node.js – Backend runtime.
• Express.js – Backend framework.
• MySQL – Database management.
• Git – Version control.
• Docker – Containerization.
• Docker Compose – Multi-container management.

6.4 Documentation Improvements

The documentation can be improved by:

• Adding screenshots of application execution.
• Adding API documentation.
• Providing troubleshooting instructions.
• Adding detailed database documentation.
• Adding contribution guidelines.
• Providing testing instructions.
• Adding deployment instructions.

7. CODE REVIEW FINDINGS AND RECOMMENDATIONS

7.1 Overall Findings

The code review shows that the College Administration and Management System follows a structured and modular approach. The separation of frontend, backend, and database components improves maintainability.

The repository is clearly organized, and Git provides effective version control. Docker improves portability and simplifies deployment.

7.2 Strengths

The major strengths are:

• Well-organized repository.
• Modular application structure.
• Meaningful Git commits.
• Feature-based branching.
• Separation of frontend and backend.
• Docker containerization.
• Clear documentation.
• Functional testing.
• Scalable architecture.

7.3 Recommendations

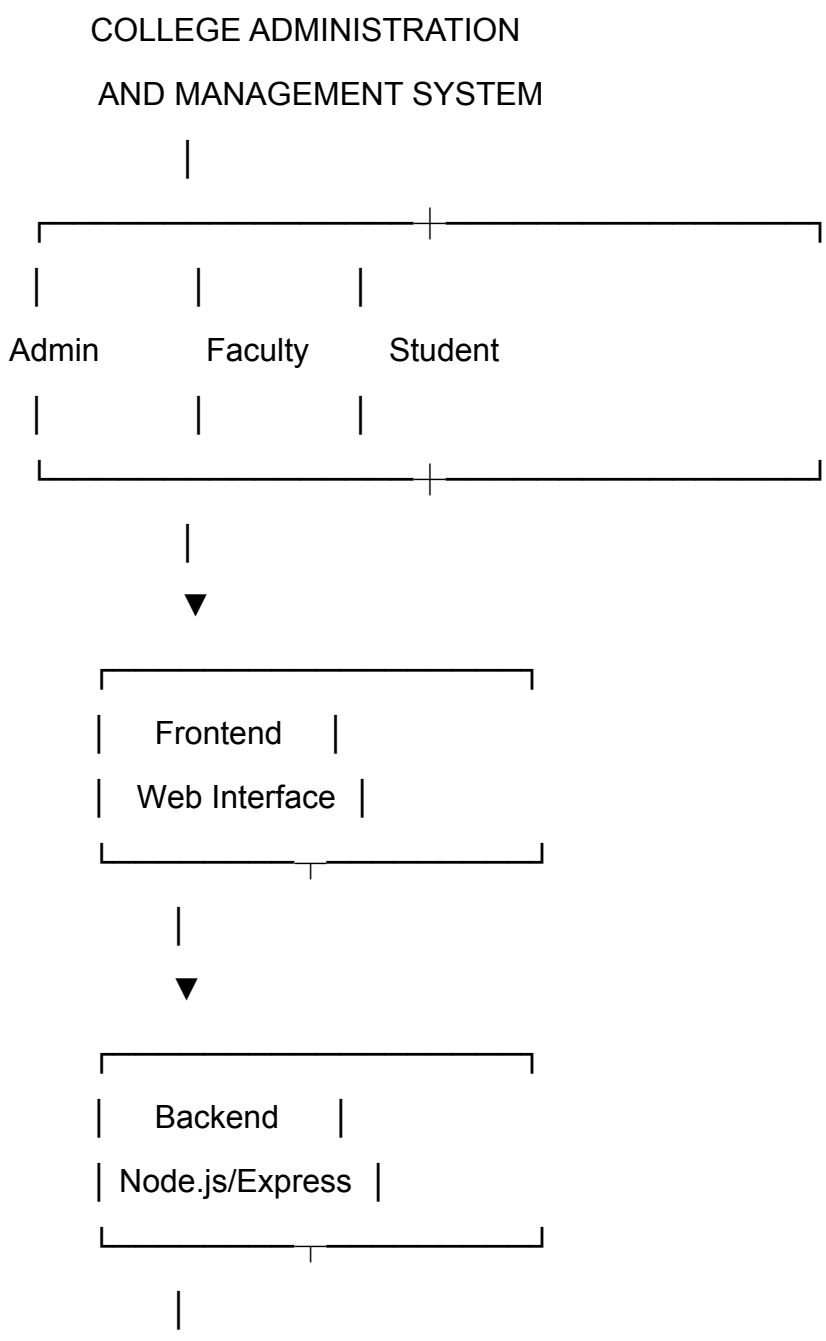
The following recommendations can improve the project:

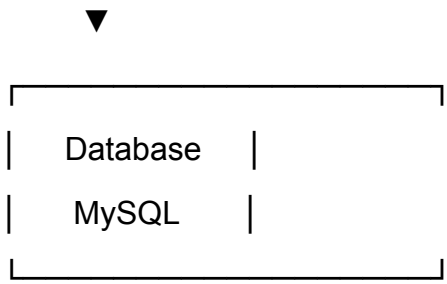
1. Implement automated unit and integration testing.
2. Add CI/CD pipelines.
3. Improve authentication and authorization.
4. Implement stronger input validation.
5. Add centralized error handling.

6. Improve database security.
7. Use secure environment variables.
8. Add API documentation.
9. Implement automated database backups.
10. Add application monitoring and logging.

8. ARCHITECTURE AND COMPONENT DESIGN

8.1 System Architecture





8.2 Component Design

The major components of the system are:

- User Authentication Component
- Student Management Component
- Faculty Management Component
- Admission Management Component
- Course Management Component
- Attendance Management Component
- Fee Management Component
- Report Management Component
- Notification Component
- Database Component

9. COMPONENT INTERACTION

The components interact with each other through the following process:

User

↓

Frontend

↓

API Request

↓

Backend

↓

Controller

↓

Business Logic

↓

Database Model

↓

MySQL Database

↓

Response

↓

Frontend

↓

User

For example, when an administrator adds a new student, the frontend sends the student's information to the backend. The backend validates the information and stores it in the database. The result is then returned to the frontend and displayed to the administrator.

10. QUALITY ATTRIBUTE ANALYSIS

Quality Attribute	Implementation
Performance	Efficient API and database operations
Security	Authentication, authorization, and input validation
Scalability	Modular architecture and Docker containerization
Maintainability	Organized repository and modular code
Reliability	Validation and error handling
Portability	Docker containers
Usability	Simple and organized user interface
Availability	Container-based deployment
Testability	Modular components and test cases